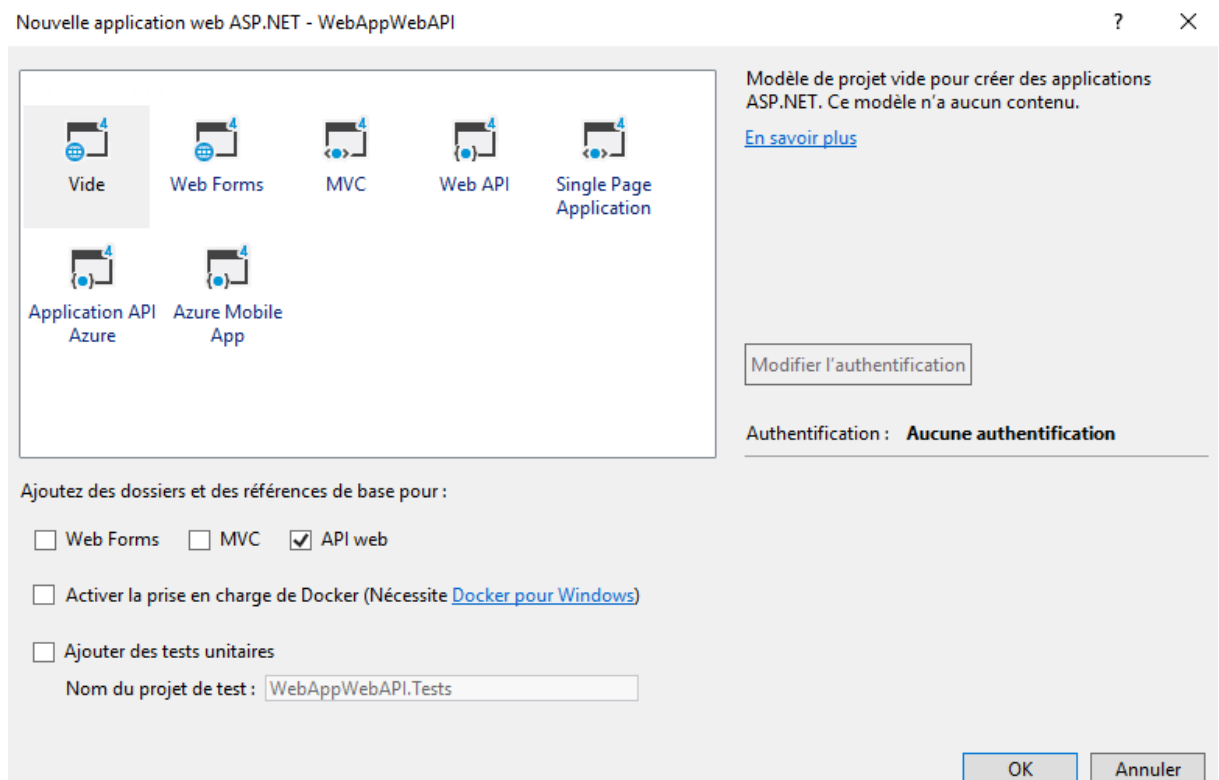


Atelier web API

I. WEB API

Dans cet atelier, vous allez créer un projet web qui expose une web API qui retour la liste des salaries, une page Web utilise jQuery pour afficher la liste des salaries comme résultat.

1. Il faut créer un projet vide de type : Application Web asp.Net (.Net Framework)



2. Par la suite, nous devons ajouter un modèle au niveau de notre solution, le modèle représente notre classe Salarie avec l'ensemble des attributs comme :

Nom, Prénom, Salaire, Fonctionne et Adresse.

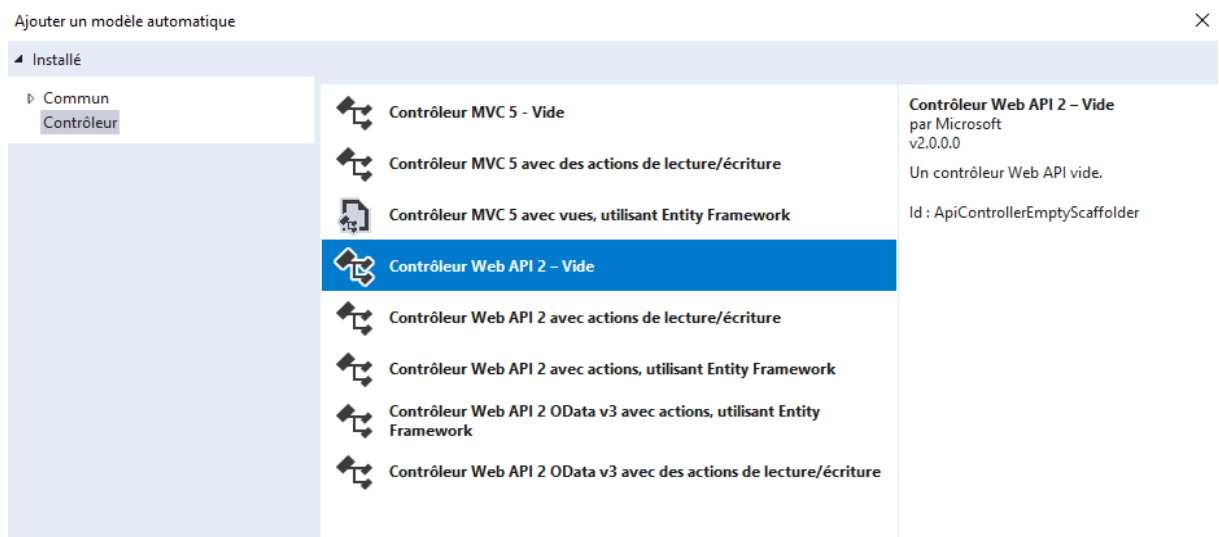
Il ne faut pas oublier d'ajouter un identifiant au niveau de la classe Salarie.

Votre modèle doit ressembler à cela :

```
public class Salarie
{
    public Int32 Id { get; set; }
    public String Name { get; set; }
```

```
public String Prenom { get; set; }
public Decimal Salaire { get; set; }
public String Fonction { get; set; }
}
```

3. Par la suite, nous devons créer la classe SalarieContext (la classe qui hérite de la classe DbContext), pour le faire, d'abord, il faut commencer par l'installation de Entity Framework.
4. Au niveau de la classe Context, rajoutez un constructeur avec la chaine de connexion Et les attributs de type DbSet qui représente nos entités(Salarie).
5. Une fois notre modèle est finalisé, nous devons rajouter un contrôleur, cela identique aux contrôleurs MVC sauf que là avec un petit changement qu'on va découvrir à la fin de cet atelier.
6. Le contrôleur que nous allons rajouter est un contrôleur de type Web API « Contrôleur Web API 2 – Vide »



7. N'oubliez pas les règles de nommage des contrôleurs, un contrôle doit porter le nom du modèle + le mot « Contoller », pour le cas de notre classe Salarie, ça sera : « SalarieController »
8. Notez que la classe SalarieController hérite de la classe « ApiController ».

Une fois nous avons le modèle, le contrôleur et la classe DbContext finalisées, nous allons commencer à implémenter les méthodes au niveau du contrôleur, l'idée est de rajouter deux méthodes, la première retourne la liste des Salaries, et la deuxième un Salarié à partir de son Id.

9. La première méthode « GetAllSalarie » doit ressembler à :

```

    public IList<Salarie> GetAllSalarie()
    {
        SalarieContext dbcontext = new SalarieContext();
        return dbcontext.Salaries.ToList();
    }

```

10. La deuxième méthode « GetSalarie » doit ressembler à :

```

    public IHttpActionResult GetSalarie(int id)
    {
        SalarieContext dbcontext = new SalarieContext();
        var Salarie = dbcontext.Salaries.FirstOrDefault((p) => p.Id == id);
        if (Salarie == null)
        {
            return NotFound();
        }
        return Ok(Salarie);
    }

```

Une fois les méthodes sont créées nous pouvons tester le fonctionnement des Web API à partir d'un code JQuery et JavaScript qu'on va implémenter dans une page HTML.

11. Ajouter une page HTML nommée « ListeSalarie » pour charger la liste des Salaries et rechercher un Salarie à partir de son Id, votre code doit ressembler au code ci-dessous :

```

<body>
  <div>
    <h2>Liste Salaries</h2>
    <ul id="Salaries" />
  </div>
  <div>
    <h2>Rechercher un Salarie à partir de son Id</h2>
    <input type="text" id="SalarieId" size="5" />
    <input type="button" value="Search" onclick="Rechercher();" />
    <p id="Salarie" />
  </div>

  <script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-2.0.3.min.js"></script>
  <script>
    var uri = 'api/Salarie';

    $(document).ready(function () {
      // Envoyer une requete JSON
      $.getJSON(uri)
        .done(function (data) {
          // En cas de retour d'une liste de salarie
          $.each(data, function (key, item) {
            // Ajouter les salaries dans le liste des items
            $('<li>', { text: item.Id + ' : ' + item.Name + ' ' +
              item.Prenom}).appendTo($('#Salaries'));
          });
        });
    });
  </script>

```

```
});  
});
```

```
function formatItem(item) {  
    return item.Name + ' : ' + item.Prenom + ' ' + item.Salaire + ' €';  
}  
  
function Rechercher() {  
    var id = $('#SalarieId').val();  
    $.getJSON(uri + '/' + id)  
        .done(function (data) {  
            $('#Salarie').text(formatItem(data));  
        })  
        .fail(function (jqXHR, textStatus, err) {  
            $('#Salarie').text('Erreur : ' + err);  
        });  
}  
</script>  
</body>
```

II. WEB API

Après la création de la page de recherche, nous allons maintenant créer une interface Web HTML avec les fonctions d'ajout, suppression, modification.

12. Pour faire cette deuxième partie, on va générer automatiquement un Contrôleur à partir de la classe modèle « Salarie ».

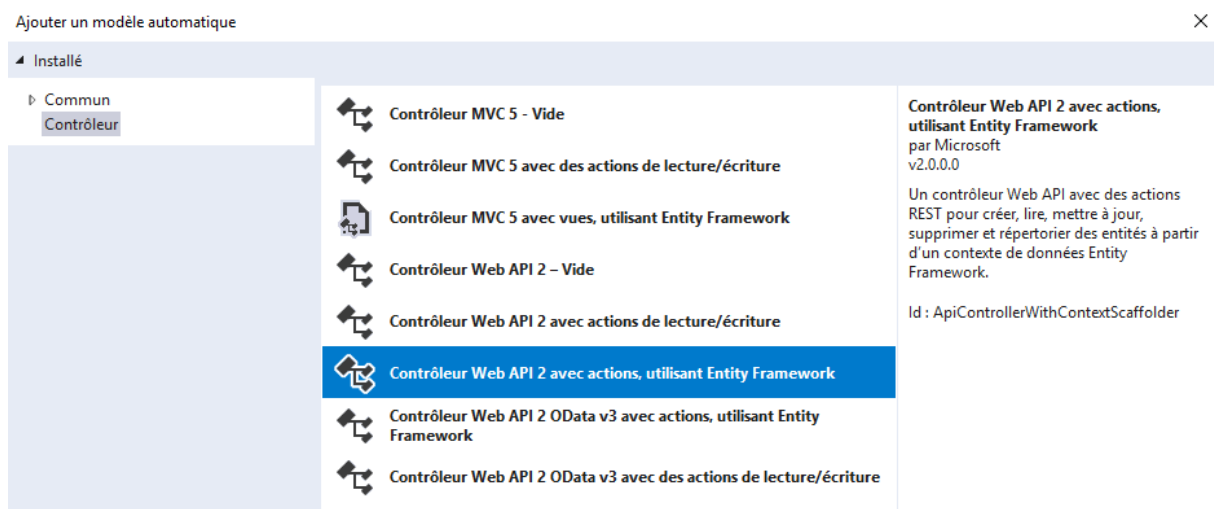
13. Cliquez avec le bouton droit sur le dossier *Contrôleurs*.

14. Sélectionnez **Ajouter > Nouvel élément généré automatiquement**.

15. Sélectionnez **Contrôleur d'API avec actions, utilisant Entity Framework**, puis **Ajouter**.

16. Dans la boîte de dialogue **Contrôleur d'API avec actions, utilisant Entity Framework** :

- Sélectionnez TodoItem (TodoApi. Models) dans la classe de modèle.
- Sélectionnez TodoContext (TodoApi. Models) dans la classe de contexte de données.
- Sélectionnez Ajouter .



17. Examinez le code des méthodes générées dans le contrôleur Web API.

18. Nous allons maintenant rajouter une page pour les CRUD des salaires (Ajout, modification, et suppression).
19. Ajoutez une page HTML « Gestion Salarie ».
20. Au niveau de cette page, ajoutez 2 zones textes pour l'ajout d'un nouveau salaire avec son nom et prénom et 3 boutons un pour l'ajout, un pour la modification et un dernier pour la suppression.
21. Le code de votre page va être similaire au code ci-dessous :

```
<body>
<script src="Scripts/jquery-3.2.1.min.js"></script>
<script>
    $(document).ready(Load());
    function Load() {
        var url = 'api/Salaries';
        $.getJSON(url)
            .done(function (data) {
                $.each(data, function (key, item) {
                    $('<li>', { text: item.Id + ':' + item.Name + ' ' +
item.Prenom }).appendTo($('#ListeSalarie'));
                });
            });
    }

    function Find() {
        var url = 'api/Salaries/' + $('#IdSalarie').val();
        $.getJSON(url)
            .done(function (data) {
                $('#FisrtNameText').val(data.Prenom);
                $('#LastNameText').val(data.Name);
            });
    }

    function Add() {
        var url = 'api/Salaries';
        var salarie = {
            Prenom: $('#FisrtNameText').val(),
            Name: $('#LastNameText').val()
        };

        $.ajax({
            type: 'POST',
            data: JSON.stringify(salarie),
            url: url,
            contentType: 'application/json'
        });
    }

    function MAJ() {
        var url = 'api/Salaries/' + $('#IdSalarie').val();
        var salarie = {
            Id: $('#IdSalarie').val(),
            Prenom: $('#FisrtNameText').val(),
            Name: $('#LastNameText').val()
        };
    }

```

```

        $.ajax({
            type: 'PUT',
            data: JSON.stringify(salarie),
            url: url,
            contentType: 'application/json'
        });
    }

    function Del() {
        var url = 'api/Salaries/' + $('#IdSalarie').val();
        $.ajax({
            type: 'DELETE',
            url: url,
            contentType: 'application/json'
        });
    }
}

</script>
<div>
    <h2>Liste des salaries</h2>
    <ul id="ListeSalarie"></ul>
</div>
<div>
    <input type="text" id="IdSalarie" />
    <input type="button" value="Rechercher" onclick="Find();" />
</div>
<table>
    <tr>
        <td>
            <input type="text" id="FisrtNameText" />
        </td>
        <td>
            <input type="text" id="LastNameText" />
        </td>
    </tr>
    <tr>
        <td>
            <input type="button" value="Ajouter" onclick="Add(); Load();" />
        </td>
        <td>
            <input type="button" value="Modification" onclick="MAJ(); Load();" />
        </td>
        <td>
            <input type="button" value="Suppression" onclick="Del(); Load();" />
        </td>
    </tr>
</table>
</body>

```